

```

1  //
  =====
2  //  FILE: HUDPanelFinalizer_Manager.cs
3  //  PATH: Engine/UI/HUDPanels/
4  //  MODULE: HUD Panel Finalizer Manager (Controlling Program)
5  //
6  //  ROLE:
7  //      Central orchestrator for the CASH HUD panel finalization pipeline. Coordinates
  manual override
8  //      control, color parsing, geometry/crosshair resolution, and UIState generation.
  Acts as the
9  //      controlling program that wires all smaller Finalizer modules together.
10 //
11 //  RESPONSIBILITIES:
12 //      - Own the lifecycle of HUDPanelFinalizer_Control, _ColorParser, _Resolver, and
  _UIStateBuilder.
13 //      - Load persisted configuration via HUDConfigManager and distribute it to
  submodules.
14 //      - Invoke manual override processing and color parsing in deterministic order.
15 //      - Resolve final geometry, colors, and crosshair configuration for HUDPanel_Cash.
16 //      - Produce UIState objects for the GPU UI pipeline via the UIStateBuilder.
17 //      - Emit DiagnosticEntry messages for all orchestration steps.
18 //
19 //  NON-RESPONSIBILITIES:
20 //      - Direct rendering of the HUD panel (handled by HUDPanel_Cash).
21 //      - Low-level color parsing logic (delegated to HUDPanelFinalizer_ColorParser).
22 //      - Manual override field management (delegated to HUDPanelFinalizer_Control).
23 //      - Geometry/crosshair normalization (delegated to HUDPanelFinalizer_Resolver).
24 //      - UIState element construction (delegated to HUDPanelFinalizer_UIStateBuilder).
25 //
26 //  ARCHITECTURAL NOTES:
27 //      - This manager is the single entry point for the CASH HUD finalization pipeline.
28 //      - All submodules are composed here to maintain deterministic ordering and
  traceability.
29 //      - Designed to be extensible for additional HUD panels and future Finalizer
  modules.
30 //      - Integrates with the diagnostics pipeline for full traceability across UI
  subsystems.
31 //
  =====
32 using SASZombieAssaultTD.Engine.Diagnostics;
33 using SASZombieAssaultTD.Engine.UI.HUDPanels;
34 using IDrawingContext = SASZombieAssaultTD.Engine.Rendering.IDrawingContext;
35
36 namespace SASZombieAssaultTD.Engine.UI
37 {
38     public class HUDPanelFinalizer_Manager : IHUDElement
39     {
40         public HUDManager _hudManager;
41         public HUDPanel_CashUpdate _cashPanel;
42
43         private readonly HUDPanelFinalizer_Control _control;
44         private readonly HUDPanelFinalizer_ColorParser _colorParser;
45         private readonly HUDPanelFinalizer_Resolver _resolver;
46         private readonly HUDPanelFinalizer_UIStateBuilder _uiStateBuilder;
47
48         // Use the HUDPanels.UIState type, not Engine.UI.UIState
49         private HUDPanels.UIState _cachedState;
50
51         public string Id => "cash_panel_manager";
52         public int Layer => 998;
53
54         string IHUDElement.Id => Id;
55         int IHUDElement.Layer => Layer;
56
57         public HUDPanelFinalizer_Manager(
58             HUDManager hudManager,

```

```

59         HUDPanel_CashUpdate cashPanel,
60         HUDPanelFinalizer_Control control,
61         HUDPanelFinalizer_ColorParser colorParser,
62         HUDPanelFinalizer_Resolver resolver,
63         HUDPanelFinalizer_UIStateBuilder uiStateBuilder)
64     {
65         _hudManager = hudManager;
66         _cashPanel = cashPanel;
67         _control = control;
68         _colorParser = colorParser;
69         _resolver = resolver;
70         _uiStateBuilder = uiStateBuilder;
71
72         DLogger.Log("UI", "FinalizerManager",
73             "HUDPanelFinalizer_Manager constructed and wired to submodules.",
74             "FinalizerManager_Created", 0);
75     }
76
77     public void Initialize()
78     {
79         HUDConfigManager.LoadPanel("cash_panel", _control);
80
81         ProcessManualOverrides();
82         ResolveFinalState();
83         BuildUIState();
84     }
85
86     private void ProcessManualOverrides()
87     {
88         var controlData = _control.GetManualEntryData();
89
90         var fillResult = _colorParser.TryParse(controlData.FillColorToken);
91         var textResult = _colorParser.TryParse(controlData.TextColorToken);
92         var priceResult = _colorParser.TryParse(controlData.ItemPriceColorToken);
93
94         if (!fillResult.Success || !textResult.Success || !priceResult.Success)
95         {
96             _control.SetErrorMessage(
97                 fillResult.Error ?? textResult.Error ?? priceResult.Error
98             );
99             _control.SetManualOverrideEnabled(false);
100             return;
101         }
102
103         _resolver.SetManualColors(fillResult.Color, textResult.Color);
104         _resolver.SetItemPriceColor(priceResult.Color);
105         _control.SetManualOverrideEnabled(true);
106     }
107
108     private void ResolveFinalState()
109     {
110         var controlData = _control.GetManualEntryData();
111
112         _resolver.SetGeometry(controlData.X, controlData.Y, controlData.Width,
113             controlData.Height);
114
115         _resolver.SetCrosshair(
116             controlData.CrosshairCenterX,
117             controlData.CrosshairCenterY,
118             controlData.CrosshairArmLength,
119             controlData.CrosshairThickness,
120             controlData.CrosshairColor,
121             controlData.UseFlashColor,
122             controlData.FlashColor
123         );
124
125         _resolver.Resolve();
126     }

```

```

127     private void BuildUIState()
128     {
129         var resolved = _resolver.GetResolvedState();
130
131         _cachedState = _uiStateBuilder.Build(
132             x: resolved.X,
133             y: resolved.Y,
134             width: resolved.Width,
135             height: resolved.Height,
136             fillColor: resolved.FillColor,
137             textColor: resolved.TextColor,
138             itemPriceColor: resolved.ItemPriceColor,
139             crosshairCenterX: resolved.CrosshairCenterX,
140             crosshairCenterY: resolved.CrosshairCenterY,
141             crosshairArmLength: resolved.CrosshairArmLength,
142             crosshairThickness: resolved.CrosshairThickness,
143             crosshairColor: resolved.CrosshairColor,
144             useFlashColor: resolved.UseFlashColor,
145             flashColor: resolved.FlashColor
146         );
147     }
148
149     // HUDManager uses this to get the resolved state
150     public HUDPanelFinalizerResolvedState GetResolved()
151     {
152         return _resolver.GetResolvedState();
153     }
154
155     public void ApplyToPanel(HUDPanel_CashUpdate cashPanel,
156         HUDPanelFinalizerResolvedState resolved)
157     {
158         if (resolved.ManualOverrideEnabled)
159         {
160             cashPanel.ManualItemPriceColor = resolved.ItemPriceColor;
161
162             cashPanel.ManualFillColor = resolved.FillColor;
163             cashPanel.ManualTextColor = resolved.TextColor;
164             cashPanel.UseManualColors = true;
165             cashPanel.DisplayText = "Zero";
166         }
167         else
168         {
169             cashPanel.UseManualColors = false;
170         }
171     }
172
173     public void Update(float deltaTime)
174     {
175         _cashPanel.Update(deltaTime);
176     }
177
178     // Draw uses IDrawingContext, matching HUDPanel_CashUpdate.Draw
179     public void Draw(IDrawingContext context)
180     {
181         _cashPanel.Draw(context);
182     }
183
184     // Return the HUDPanels.UIState type
185     public HUDPanels.UIState GetUIState()
186     {
187         return _cachedState;
188     }
189
190     public void SaveConfig()
191     {
192         var resolved = _resolver.GetResolvedState();
193
194         HUDConfigManager.SetPanel(
195             "cash_panel",

```

```

195         resolved.X,
196         resolved.Y,
197         resolved.Width,
198         resolved.Height,
199         Layer
200     );
201
202     HUDConfigManager.SetCrosshair(
203         "cash_panel",
204         resolved.CrosshairCenterX,
205         resolved.CrosshairCenterY,
206         resolved.CrosshairArmLength,
207         resolved.CrosshairThickness,
208         resolved.CrosshairColor,
209         resolved.UseFlashColor,
210         resolved.FlashColor
211     );
212 }
213
214
215 public struct HUDPanelFinalizerControlData
216 {
217     public int X, Y, Width, Height;
218     public string FillColorToken, TextColorToken;
219     public string ItemPriceColorToken;
220     public int CrosshairCenterX, CrosshairCenterY, CrosshairArmLength,
221         CrosshairThickness;
222     public System.Drawing.Color CrosshairColor, FlashColor;
223     public bool UseFlashColor;
224 }
225
226 public struct HUDPanelFinalizerResolvedState
227 {
228     public int X, Y, Width, Height;
229     public Color FillColor, TextColor, ItemPriceColor;
230     public int CrosshairCenterX, CrosshairCenterY, CrosshairArmLength,
231         CrosshairThickness;
232     public System.Drawing.Color CrosshairColor, FlashColor;
233     public bool UseFlashColor;
234     public bool ManualOverrideEnabled;
235 }

```